

AAKAAR

13 - AGENT

Component: Remote Agent

The desktop/RPA worker that executes capabilities on remote machines

Component

AUDIENCE	Engineers, Operations
VERSION	1.0
STATUS	Released
CLASSIFICATION	Confidential

Remote Agent

In plain terms: the remote agent is a small, trusted worker program you install on an ordinary computer — a branch teller's PC, a back-office workstation, a kiosk. It lets Aakaar reach out and *do things on that machine*: click buttons, type into a legacy banking application, read a window title, run a command. Think of it as a careful robot hand that Aakaar can borrow on a specific machine, but only for work that machine's owner (its tenant) has explicitly authorised. The computer never opens a door for the outside world — the agent always calls *out* to Aakaar, never the other way round.

This is how Aakaar automates the last mile of banking work that lives in desktop software with no API: a green-screen core-banking terminal, a vendor reconciliation tool, a Windows-only KYC scanner. The agent runs the steps a human would, on the machine where those applications actually run.

The agent is **RPA (robotic process automation)** done safely: outbound-only networking, per-agent identity, just-in-time secrets, and a privacy-preserving activity recorder. It runs the *same* capability code the server runs, so a capability is "write once, run server-or-agent."

What it is

The agent is a Python service (`aakaar-agent`) installed on a remote/desktop machine. On startup it dials out over an authenticated WebSocket to the API's `/ws/agents` endpoint, announces what it is, and then waits for the API to dispatch tasks. Each task names a **capability ref** (e.g. `cap.desktop_click`); the agent runs that capability locally and sends the result back.

It loads two kinds of capabilities:

Capability ref	Needs a GUI session	What it does
<code>cap.desktop_click</code>	yes	click at coordinates / on an element
<code>cap.desktop_type</code>	yes	type text
<code>cap.key_send</code>	yes	send keys / hotkeys
<code>cap.desktop_scroll</code>	yes	scroll
<code>cap.clipboard_write</code>	yes	set the clipboard
<code>cap.window_manage</code>	yes	focus / move / resize windows
<code>cap.activity_recording</code>	yes	record desktop activity as a redacted trace
<code>cap.shell_exec</code>	no	run an argv command (no shell string — injection-safe)
<code>cap.system_info</code>	no	report host facts

GUI capabilities are imported **lazily** — a headless agent (a server with no logged-in desktop) loads fine and simply never advertises GUI caps. The shared headless caps (`shell_exec` , `system_info`) come from the `aakaar_caps` SDK, the very same library the server uses.

Banking example: a reconciliation workflow places a `cap.desktop_type` node and a `cap.key_send` node on the branch PC that hosts the legacy ledger app. The agent drives the app exactly as the clerk would — opens the day's batch, types the control total, presses Enter — and reports back success or the on-screen error.

Agent internals

Maximum text size in diagram exceeded